

Campus Bulletin Board System

校园论坛系统设计报告

Group 10: 陈思源、胡睿翔、叶凤林、陈诺、黄磊、吴文涛

2026 年 5 月 18 日

浙江大学

系统体系结构

数据库设计

关键过程描述

组件设计

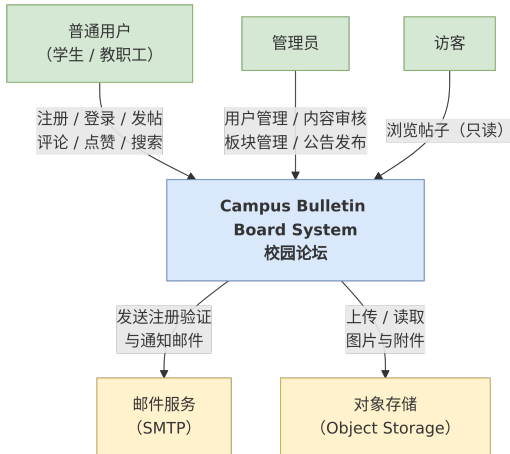
可靠性及安全性设计

用户界面设计

系统体系结构

外部交互边界

- **普通用户**：注册/登录、发帖、评论、点赞、搜索
- **管理员**：用户管理、内容审核、板块管理、公告发布
- **访客**：浏览帖子（只读）
- **外部系统**：邮件服务（SMTP）、对象存储（OSS）

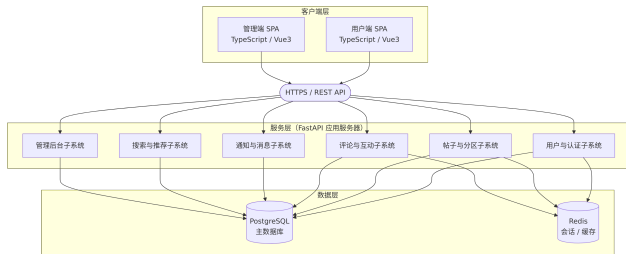


三层架构

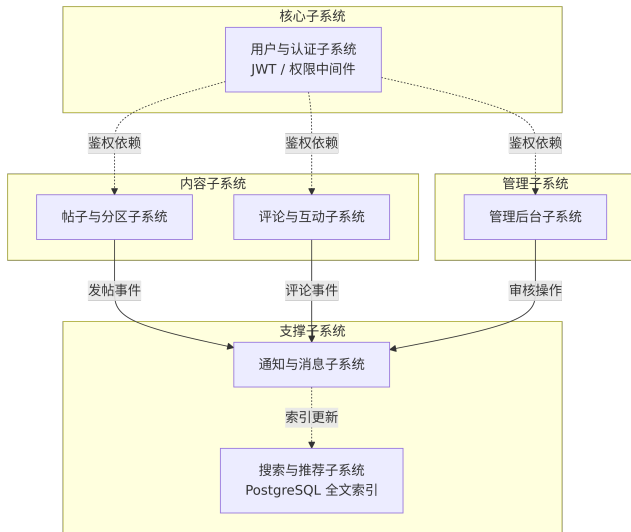
1. **客户端层**: Vue3 + TypeScript SPA
2. **服务层**: FastAPI 应用服务器
3. **数据层**: PostgreSQL + Redis

关键特点

前后端分离 ■ RESTful API ■
JWT 认证



六大子系统与交互关系



关键技术决策与部署

关键技术选型

决策	选型	理由
认证方案	JWT+Redis 黑名单	无状态扩展性好
主键策略	UUID	防 ID 枚举攻击
软删除	deleted_at	支持数据恢复 审计
搜索	PostgreSQL 全文检索 (暂定)	校园级规模足够
代码质量	black+ruff+pytest	保证代码质量

开发环境部署

- **Docker Compose**: PostgreSQL + Redis + Mailpit
- **uv**: Python 虚拟环境管理
- **uvicorn**: 开发服务器热重载
- **pnpm**: 前端包管理
- **Vite**: 前端热模块替换
- **Husky**: pre-commit 自动 lint

技术栈

后端: Python3.14 + FastAPI + SQLAlchemy + Pydantic + Alembic + pwdlib

前端: Vue3 + TypeScript + Element Plus + Vite

数据层: PostgreSQL + Redis

数据库设计

设计目标与通用约定

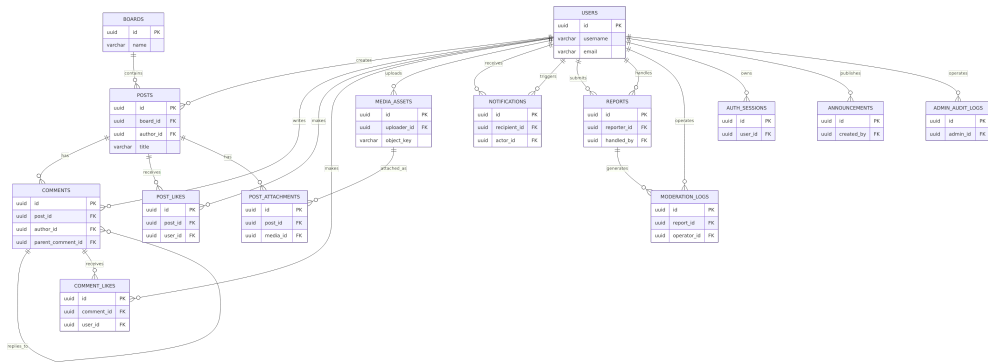
设计原则

1. **强一致优先**: 核心业务由 PostgreSQL 保证 ACID
2. **大文件分离**: 图片走对象存储, DB 仅存元数据
3. **计数器冗余**: 避免高频 COUNT 聚合
4. **软删除统一**: deleted_at, 数据可追溯

通用约定

约定	说明
主键	UUID, 防枚举
时间戳	TIMESTAMPTZ NOT NULL
软删除	deleted_at NULL
计数器	BIGINT DEFAULT 0
状态字段	VARCHAR + CHECK

实体关系概览



核心表设计要点

users — 用户表

- username VARCHAR(32), UNIQUE
- email VARCHAR(255), UNIQUE
- password_hash —pwdlib (Argon2)
- role —CHECK IN ('user','admin')
- status —CHECK IN ('active','inactive','banned')
- email_verified BOOLEAN

posts — 帖子表

- board_id / author_id —FK
- content TEXT (支持 Markdown)
- status —CHECK IN ('normal','hidden','deleted')
- is_pinned / is_featured
- like_count / comment_count / view_count

notifications — 通知表

- type CHECK IN ('comment','reply','like','system')
- is_read BOOLEAN + read_at TIMESTAMPTZ

核心索引

- `posts(board_id, is_pinned DESC, created_at DESC)`
- `notifications(recipient_id, is_read, created_at)`
- `comments(post_id, comments(parent_comment_id))`
- `post_likes(post_id, user_id) UNIQUE`
- 所有 `deleted_at` 建索引

关键设计决策

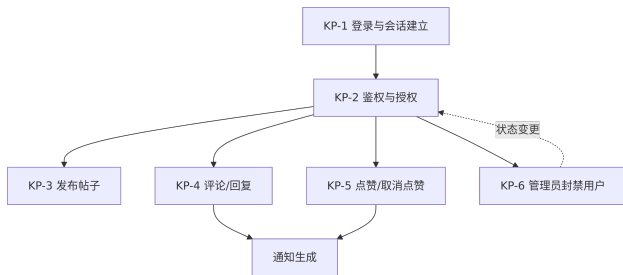
决策	理由
UUID 主键	防枚举
软删除	数据可追溯可恢复
计数器冗余	换列表免 COUNT 聚合
VARCHAR+CHECK	避免 ENUM 迁移成本
评论双外键	一次查询获取所有楼层
通知异步化	不阻塞用户请求

关键过程描述

关键过程总览

关键过程

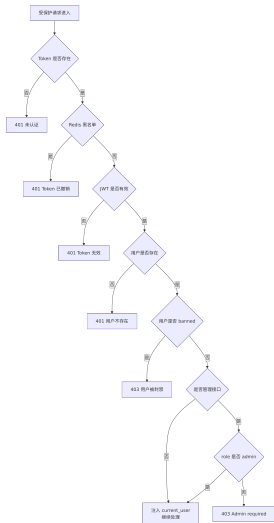
编号	过程
KP-1	登录与会话建立
KP-2	鉴权与授权
KP-3	发布帖子
KP-4	评论/回复与通知
KP-5	点赞/取消点赞
KP-6	管理员封禁用户



KP-1 & KP-2: 认证与鉴权链路

KP-1 登录流程

1. 客户端提交账号与密码
2. 按用户名或邮箱查询用户
3. `verify_password()` 校验
4. 检查 `status == active`
5. 更新 `last_login_at`
6. 生成 JWT (含 `sub`, `role`, `exp`)
7. 返回 `ApiResponse<LoginData>`



KP-3 & KP-6: 发布帖子与管理封禁

KP-3 发布帖子

流程:

1. **请求准入**: `get_current_user()` 鉴权
2. **输入校验**: Pydantic `PostCreate`
3. **数据封装**: 合并业务字段 + `author_id`
4. **持久化**: `add` → `commit` → `refresh`
5. **响应输出**: `ApiResponse<PostRead>`

异常: 未登录 401、被封禁 403、校验失败 422

KP-6 管理员封禁用户

流程:

1. **权限准入**: `require_admin()` 双重校验
2. **输入校验**: UUID 格式 + 状态正则校验
3. **实体定位**: 按 `uuid` 查询用户
4. **状态变更**: `status := "banned"`
5. **响应输出**: `AdminUserData`

效果: 登录被拒 + 现有 Token 下次请求拦截

评论/回复流程

1. 校验目标帖子存在且未被删除
2. 若 `parent_comment_id` 为空 → 创建一级评论, `root_comment_id` 指向自身
3. 若不为空 → 继承根评论编号, 创建回复
4. 同步更新 `post.comment_count` / `parent.reply_count`
5. 调用 `NotificationService` 创建通知

楼中楼树机制

一级评论: `parent_comment_id` IS NULL, `root_comment_id` 指向自身; 子回复: `parent_comment_id` 指向被回复评论, `root_comment_id` 指向根楼层

KP-5: 点赞/取消点赞与计数同步

点赞流程

1. 确认目标帖子/评论存在
2. 检查未点过 (UNIQUE 约束防重)
3. 新增 Like 记录
4. $\text{like_count} + 1$
5. 创建 like 通知

取消点赞流程

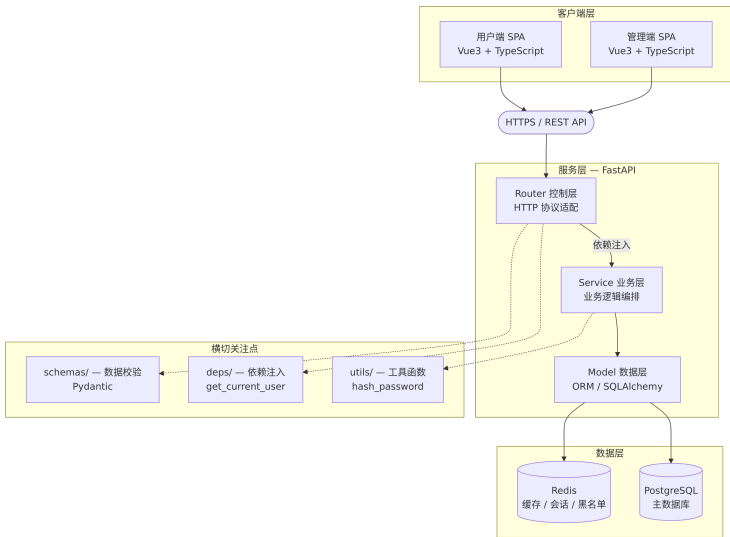
1. 定位现有点赞记录
2. 删除记录
3. $\text{like_count} - 1$ (不低于 0)

关键约束

在同一事务内更新记录 + 计数 (保证一致性) ▪ UNIQUE 约束保证幂等 ▪ 计数不得小于零

组件设计

分层架构设计



子系统内部设计元素说明（以帖子管理子系统为例）

PostService 核心方法

方法	说明
create()	构造并持久化新帖子
get_multi()	分页查询，置顶优先
get_by_id()	按主键查询，预加载
update()	部分更新 (exclude_unset)
update_special_status()	置顶/加精切换
remove()	软删除

关键关系

- **User** → **Post**: 1:N（组合弱化——删除用户不级联删帖）
- **Board** → **Post**: 1:N（帖子必须归属板块）
- **PostService** ↔ **Post**: 依赖关系（Session 注入）
- **Post** ↔ **PostStatus**: 枚举（NORMAL / HIDDEN / DELETED）

Schema 分层

PostCreate（入参）→ PostService（业务逻辑）→ PostRead（出参）

核心用例实现方案（以发布帖子为例）

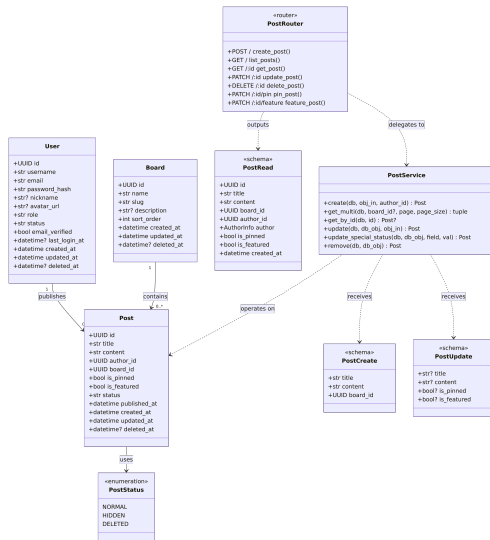
参与者、前置条件、后置条件

- **参与者**：已登录的普通用户
- **前置条件**：用户状态为 active，已选择目标板块
- **触发方式**：前端点击「发布」按钮
- **后置条件**：系统创建一条新的帖子记录，帖子关联当前用户为作者、关联指定板块为所属板，返回完整的帖子信息

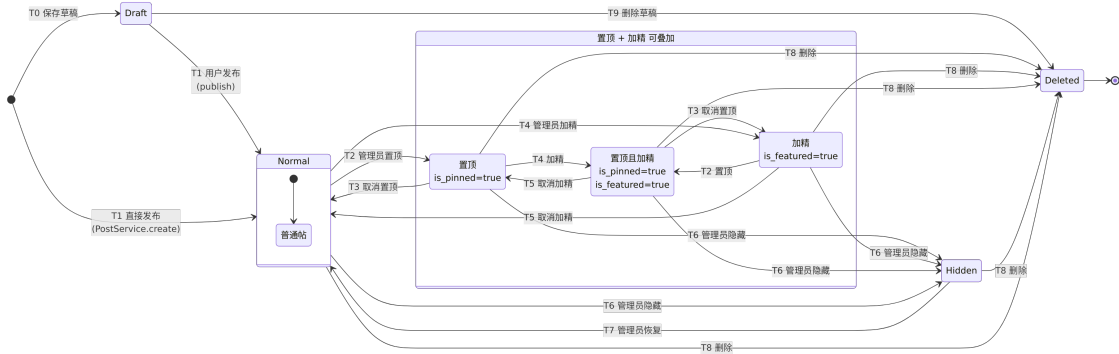
主流程

1. **请求准入**：get_current_user() 鉴权
2. **输入校验**：Pydantic PostCreate 校验 title/content/board_id
3. **数据封装**：合并业务字段 + author_id + published_at
4. **持久化**：PostService.create() → add → commit
5. **响应输出**：序列化为 ApiResponse<PostRead>

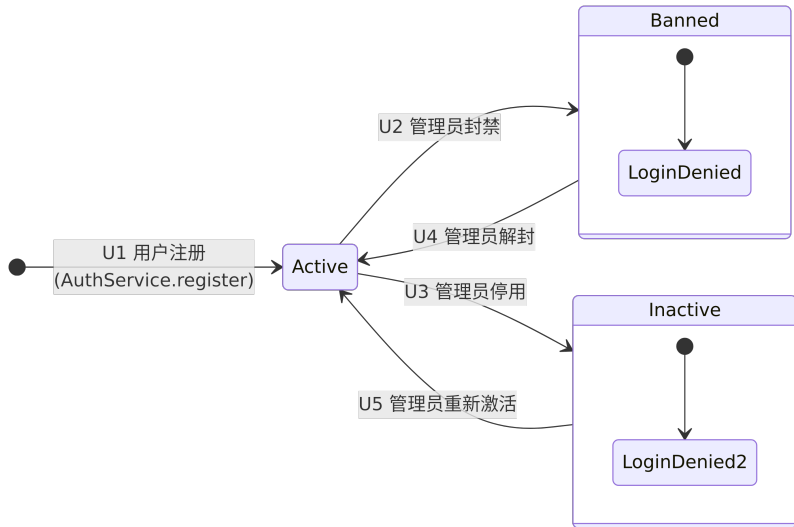
类间关系（以帖子管理子系统为例）



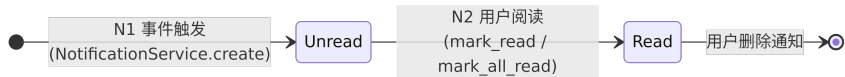
核心对象状态设计



核心对象状态设计



核心对象状态设计



独立构件设计：ORM 基础构件

独立构件（Standalone Component）是指不专属于任一业务子系统、可被多个模块复用的通用组件。

Base + IDMixin + TimestampMixin

- **Base**: SQLAlchemy DeclarativeBase
- **IDMixin**: UUID 主键，默认 `uuid.uuid4()`
- **TimestampMixin**:
 - `created_at` / `updated_at` — `TIMESTAMPZ`
 - `deleted_at` — 软删除标志

使用方式

```
class User(Base,  
            IDMixin, TimestampMixin)
```

所有实体类统一继承，保证主键、时间戳、软删除语义一致。

复用范围：users、boards、posts、comments、notifications 等全部 15 张表

核心函数

- `hash_password(plain)`
→ 返回 Argon2 加盐哈希
- `verify_password(plain, hash)`
→ 返回 bool

安全策略

- 算法：pwdlib 推荐 (Argon2)
- 不存储明文：仅保存 `password_hash`
- 自动加盐：每次哈希随机盐值
- 不可逆向：无法从哈希还原密码

复用场景

用户注册 ▪ 用户登录 ▪ 密码重置 ▪ 管理员强制改密

独立构件设计：统一响应封装

ApiResponse<T>

字段	说明
code	HTTP 状态码
message	提示信息
data	泛型业务数据
request_id	链路追踪 ID

派生类型

类型	用途
PaginatedResponse	列表分页返回
ErrorResponse	统一错误返回

设计约束

所有 Router 必须返回 ApiResponse 或其派生类型；禁止直接返回裸模型对象，保证前后端契约一致。

ORM 映射与持久化策略

ORM 映射策略

OO 概念	关系型映射	ORM 实现
实体类	数据库表	<code>class Entity(Base, Mixins)</code>
对象属性	表列	<code>Mapped[type] + mapped_column</code>
类关联 (1:N)	外键	<code>ForeignKey + relationship()</code>
继承 (Mixin)	列组合复用	Python 多重继承
枚举属性	CHECK 约束	<code>CheckConstraint</code>

事务管理

- 每次 HTTP 请求独立 Session
- Service 方法内显式 commit
- 异常时 FastAPI 自动 rollback

迁移管理 (Alembic)

- 所有 schema 变更通过迁移脚本
- 迁移必须可逆 (upgrade / downgrade)
- 启动时自动执行 `alembic upgrade head`

可靠性及安全性设计

数据一致性保证

1. **事务原子性**：点赞 = 写入记录 + 更新计数同时成功/回滚
2. **幂等约束**：UNIQUE(post_id, user_id) 防重
3. **原子更新计数**：数据库表达式更新，约束 ≥ 0
4. **软删除保护**：不物理删除，支持恢复与审计
5. **外键完整性**：帖子必须关联存在的板块和作者

服务可用性策略

1. **健康检查**：pg_isready + redis-cli ping
2. **Redis 故障降级**：缓存查询回退 PostgreSQL
3. **连接池控制**：设置大小、超时、回收时间
4. **超时与重试**：查询允许短重试，写入不盲目重试
5. **缓存策略**：更新 DB 后删缓存、空值防穿透

身份认证

- **密码存储**：pwdlib (Argon2) 加盐哈希
- **JWT 令牌**：含 sub/role/exp，访问受保护接口必须携带
- **会话管理**：Redis 黑名单，登出/封禁即时吊销
- **登录保护**：连续失败限制 (429)，防暴力破解

三层权限控制

层级	机制
接口级鉴权	校验 JWT；管理接口校验 admin
资源级鉴权	用户仅操作自己的内容
状态级鉴权	banned 禁止一切操作

角色	权限范围
访客	浏览公开帖子
普通用户	发帖/评论/点赞/举报
管理员	用户/板块/公告管理 + 审核

输入校验与注入防护

- **Pydantic 校验**：所有入参类型/长度/格式校验
- **SQL 注入防护**：SQLAlchemy ORM 参数化查询
- **XSS 防护**：服务端白名单过滤 HTML
- **内容长度限制**：标题/评论等字段上限
- **错误信息脱敏**：不暴露 SQL、路径、堆栈

文件上传安全

- **类型白名单**：png / jpeg / webp / pdf
- **大小限制**：头像/帖子图片/附件分别设置上限
- **文件名安全**：存储 Key 使用 UUID/哈希
- **哈希去重**：sha256 校验，避免重复存储
- **访问控制**：私有文件生成临时访问链接

备份与恢复

- PostgreSQL 每日完整备份
- 重要版本迭代前手动备份
- 每个迭代后至少一次恢复验证
- 对象存储元数据与文件一致性校验

敏感数据保护

- 密钥通过环境变量注入，不写业务代码
- password_hash 不出现在 API 响应/日志中
- 生产环境统一 HTTPS 加密传输
- 应用数据库账号最小权限原则
- 日志对邮箱/Token/IP 进行脱敏

日志与监控体系

- 业务日志：登录/注册/发帖/评论/点赞/举报/封禁/公告
- 审计日志：管理员操作 → admin_audit_logs；举报处理 → moderation_logs

用户界面设计

设计目标

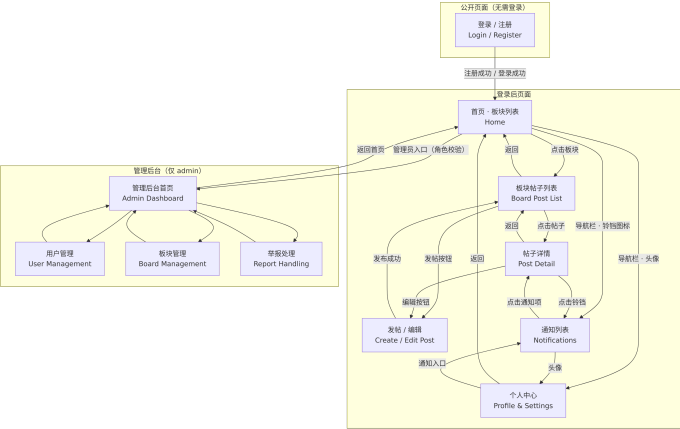
- **一致性**：组件复用率 $\geq 70\%$
- **可用性**：核心任务 ≤ 3 步完成
- **可访问性**：WCAG AA，色彩对比度 $\geq 4.5:1$

前端技术栈

层	技术	选型理由
框架	Vue 3 + TypeScript	响应式成熟、TS 完备
构建	Vite	热更新 $< 1s$
UI 库	Element Plus	中文生态友好
路由	Vue Router 4	导航守卫、懒加载
状态管理	Pinia	TS 友好
HTTP	Axios	拦截器、超时控制
富文本	Tiptap	ProseMirror、Vue3 集成

设计原则：状态驱动视图 ▪ 关注点分离（SFC） ▪ 移动优先 ▪ 最少惊讶 ▪ 容错与降级

页面流转与核心页面设计



全局交互模式

- **即时校验**：表单失焦时校验
- **防抖输入**：搜索/唯一性校验
300-500ms debounce
- **乐观更新**：点赞先改 UI → 发 API
→ 失败回滚
- **确认操作**：删除/封禁弹出确认框
- **离开确认**：未保存内容
beforeRouteLeave
- **骨架屏**：首次加载避免全局 spinner
- **键盘快捷键**：Ctrl+Enter 提交、Esc
关闭弹窗

视觉设计 Token

- **品牌色**：#409eff (主色)、#f56c6c
(危险)、#67c23a (成功)
- **排版层级**：H1 28px / H2 24px / H3
18px / Body 14px
- **间距**：4/8/16/24/32/48px 递增
- **圆角**：sm 4px / md 8px / lg 12px
- **响应式断点**：< 768px / 768-1023px /
≥ 1024px
- **内容最大宽度**：1200px 居中
- **动画时长**：页面 200ms / 弹窗 300ms /
列表 300ms

谢谢观看 Q.A.